

1. Análisis del Iterador suspenso

El primer fragmento de código nos pide ejecutar `suspenso(X + Y + Z, [X, Y, Z])` con las constantes $X=2$, $Y=7$ y $Z=4$.

- **Constantes:** $X=2$, $Y=7$, $Z=4$.
- **Llamada inicial:** `suspenso(2 + 7 + 4, [2, 7, 4])` -> `suspenso(13, [2, 7, 4])`.

A continuación se muestra la traza de ejecución paso a paso, mostrando cada marco de pila (cada llamada recursiva) y lo que imprime (yield en Python se traduce como "producir" o "entregar" un valor).

Traza de Ejecución:

1. **Llamada: `suspenso(a: 13, b: [2, 7, 4])`**
 - La lista `b` no está vacía.
 - Se produce `a + b[0]` -> `13 + 2` -> **Produce 15**.
 - Se llama recursivamente con `suspenso(b[0], b[1:])` -> `suspenso(2, [7, 4])`.
2. **Llamada: `suspenso(a: 2, b: [7, 4])`**
 - La lista `b` no está vacía.
 - Se produce `a + b[0]` -> `2 + 7` -> **Produce 9**.
 - Se llama recursivamente con `suspenso(b[0], b[1:])` -> `suspenso(7, [4])`.
3. **Llamada: `suspenso(a: 7, b: [4])`**
 - La lista `b` no está vacía.
 - Se produce `a + b[0]` -> `7 + 4` -> **Produce 11**.
 - Se llama recursivamente con `suspenso(b[0], b[1:])` -> `suspenso(4, [])`.
4. **Llamada: `suspenso(a: 4, b: [])`**
 - La lista `b` está vacía.
 - Se produce `a` -> **Produce 4**.
 - Termina la recursión.

Salida Impresa:

El bucle for x in ... print(x) imprimirá cada valor producido en orden:

15

9

11

4

2. Análisis del Iterador misterio

Esta parte nos pide ejecutar for x in misterio(5): print(x).

Analicemos la lógica:

- **suspenso(0, x):** La traza anterior nos mostró que suspenso(a, b) produce sumas de pares adyacentes. Si $a = 0$, suspenso(0, [b0, b1, b2]) producirá b0, b0+b1, b1+b2 y b2.
 - suspenso(0, [1]) -> Produce 0+1 (1), luego suspenso(1, []) -> Produce 1. **Salida: 1, 1.**
 - suspenso(0, [1, 1]) -> Produce 0+1 (1), luego suspenso(1, [1]) -> Produce 1+1 (2), 1. **Salida: 1, 2, 1.**
 - suspenso(0, [1, 2, 1]) -> Produce 0+1 (1), suspenso(1, [2, 1]) -> Produce 1+2 (3), suspenso(2, [1]) -> Produce 2+1 (3), 1. **Salida: 1, 3, 3, 1.**
- **misterio(n):** Este iterador genera la **fila n-ésima del triángulo de Pascal** (comenzando en n=0). Cada llamada a misterio(n) usa la fila n-1 y la pasa por suspenso(0, ...) para generar la fila n.

Traza de Ejecución de misterio(5):

1. **Llamada: misterio(5)**
 - Llama a misterio(4).
2. **Llamada: misterio(4)**
 - Llama a misterio(3).

3. Llamada: misterio(3)

- Llama a misterio(2).

4. Llamada: misterio(2)

- Llama a misterio(1).

5. Llamada: misterio(1)

- Llama a misterio(0).

6. Llamada: misterio(0)

- n es 0.
- **Produce [1].**

7. Retorno a misterio(1):

- x toma el valor [1].
- r se inicializa como [].
- Bucle for y in suspenso(0, [1]):
 - y es 1. r se vuelve [1].
 - y es 1. r se vuelve [1, 1].
- **Produce [1, 1].**

8. Retorno a misterio(2):

- x toma el valor [1, 1].
- r se inicializa como [].
- Bucle for y in suspenso(0, [1, 1]):
 - y es 1. r se vuelve [1].
 - y es 2. r se vuelve [1, 2].
 - y es 1. r se vuelve [1, 2, 1].
- **Produce [1, 2, 1].**

9. Retorno a misterio(4):

- Produce [1, 3, 3, 1]. (Esto es un error en mi traza manual, debería ser [1, 4, 6, 4, 1])
- *Corrección:* misterio(3) produce [1, 3, 3, 1].

- **Retorno a misterio(4):**
 - x es [1, 3, 3, 1].
 - suspenso(0, [1, 3, 3, 1]) produce 1, 4, 6, 4, 1.
 - r se vuelve [1, 4, 6, 4, 1].
- **Produce [1, 4, 6, 4, 1].**

10. Retorno a misterio(5):

- x es [1, 4, 6, 4, 1].
- r se inicializa como [].
- Bucle for y in suspenso(0, [1, 4, 6, 4, 1]):
 - suspenso produce 1, 5, 10, 10, 5, 1.
 - r se construye como [1, 5, 10, 10, 5, 1].
- **Produce [1, 5, 10, 10, 5, 1].**

Salida Impresa:

El bucle for x in misterio(5) solo recibe el valor final producido por misterio(5).

[1, 5, 10, 10, 5, 1]